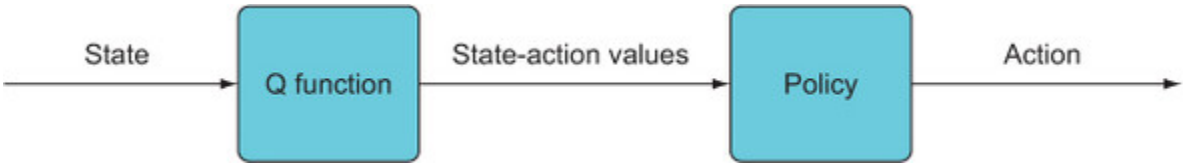
A normal Q function is a function $Q(s,a)$: $S \times A \rightarrow R$ (figure 9.4); it's a function from a state-action pair to a reward (some real number). We can remedy the problems with IL-Q by making a slightly more sophisticated Q function that incorporates knowledge of the actions of other agents, $Q_j(s,a_j,a_{-j})$: $S \times A_j \times A_{-j} \rightarrow R$. This is a Q function for the agent indexed by j that takes a tuple of the state, agent j's action, and all the other agents' actions (denoted $-j$, pronounced "not j") to the predicted reward for this tuple (again, just a real number). It is known that a Q function of this sort regains the convergence guarantee that it will eventually learn the optimal value and policy functions, and thus this modified Q function is able to perform much better.

Figure 9.4. The Q function takes a state and produces state-action values (Q values), which are then used by the policy function to produce an action. Alternatively, we can directly train a policy function that operates on a state and returns a probability distribution over actions.



Unfortunately, this new Q function is intractable when the number of agents is large because the joint action-space a_{-j} is extremely large and grows exponentially with the number of agents. Remember how we encode an action? We use a vector with length equal to the number of actions. If we want to encode a single action, we make this a *one-hot vector* where all elements are 0 except at the position corresponding to the action, which is set to 1. For example, in the Gridworld environment the agent has four actions (up, down, left, right), so we encode actions as a length 4 vector, where [1,0,0,0] could be encoded as "up" and [0,1,0,0] could be "down" and so forth.

Remember, the policy $\pi(s)$:$S \rightarrow A$ is a function that takes a state and returns an action. If it is a deterministic policy, it will have to return one of these one-hot vectors; if it is a stochastic policy, it returns a probability distribution over the actions, e.g., [0.25,0.25,0.2,0.3]. The exponential growth is due to the fact that if we want to unambiguously encode a joint action—for example, the joint action of two agents with four actions each in